

Matplotlib for beginners

Matplotlib is a library for making 2D plots in Python. It is designed with the philosophy that you should be able to create simple plots with just a few commands:

1 Initialize

```
import numpy as np
import matplotlib.pyplot as plt
```

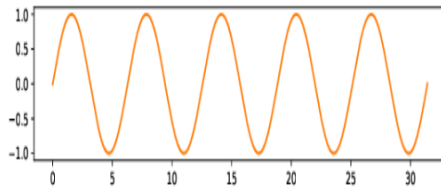
2 Prepare

```
X = np.linspace(0, 10*np.pi, 1000)
Y = np.sin(X)
```

3 Render

```
fig, ax = plt.subplots()
ax.plot(X, Y)
plt.show()
```

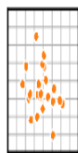
4 Observe



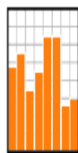
Choose

Matplotlib offers several kind of plots (see Gallery):

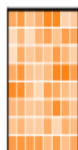
```
X = np.random.uniform(0, 1, 100)
Y = np.random.uniform(0, 1, 100)
ax.scatter(X, Y)
```



```
X = np.arange(10)
Y = np.random.uniform(1, 10, 10)
ax.bar(X, Y)
```



```
Z = np.random.uniform(0, 1, (8, 8))
ax.imshow(Z)
```



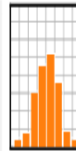
```
Z = np.random.uniform(0, 1, (8, 8))
ax.contourf(Z)
```



```
Z = np.random.uniform(0, 1, 4)
ax.pie(Z)
```



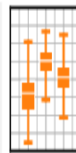
```
Z = np.random.normal(0, 1, 100)
ax.hist(Z)
```



```
X = np.arange(5)
Y = np.random.uniform(0, 1, 5)
ax.errorbar(X, Y, Y/4)
```



```
Z = np.random.normal(0, 1, (100, 3))
ax.boxplot(Z)
```



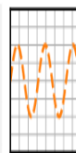
Tweak

You can modify pretty much anything in a plot, including limits, colors, markers, line width and styles, ticks and ticks labels, titles, etc.

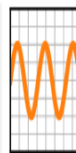
```
X = np.linspace(0, 10, 100)
Y = np.sin(X)
ax.plot(X, Y, color="black")
```



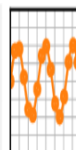
```
X = np.linspace(0, 10, 100)
Y = np.sin(X)
ax.plot(X, Y, linestyle="--")
```



```
X = np.linspace(0, 10, 100)
Y = np.sin(X)
ax.plot(X, Y, linewidth=5)
```



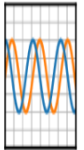
```
X = np.linspace(0, 10, 100)
Y = np.sin(X)
ax.plot(X, Y, marker="o")
```



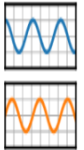
Organize

You can plot several data on the same figure, but you can also split a figure in several subplots (named Axes):

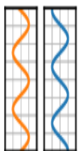
```
X = np.linspace(0, 10, 100)
Y1, Y2 = np.sin(X), np.cos(X)
ax.plot(X, Y1, X, Y2)
```



```
fig, (ax1, ax2) = plt.subplots(2, 1)
ax1.plot(X, Y1, color="C1")
ax2.plot(X, Y2, color="C0")
```

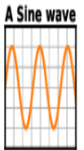


```
fig, (ax1, ax2) = plt.subplots(1, 2)
ax1.plot(Y1, X, color="C1")
ax2.plot(Y2, X, color="C0")
```



Label (everything)

```
ax.plot(X, Y)
fig.suptitle(None)
ax.set_title("A Sine wave")
```



```
ax.plot(X, Y)
ax.set_ylabel(None)
ax.set_xlabel("Time")
```



Explore

Figures are shown with a graphical user interface that allows to zoom and pan the figure, to navigate between the different views and to show the value under the mouse.

Save (bitmap or vector format)

```
fig.savefig("my-first-figure.png", dpi=300)
fig.savefig("my-first-figure.pdf")
```

Matplotlib: for visualizations

What is matplotlib?

A python library used to create static, animated and interactive visualizations.

key features:

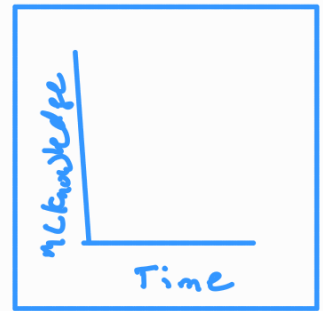
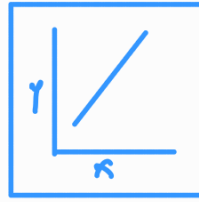
- i) versatility: supports a variety of plot types including line, scatter bar, pie charts, more complex visualizations like 3D plot.
- ii) customization: every aspect of plot can be customized.
- iii) integration: works well with Numpy, Pandas and Seaborn
- iv) Interactive plots: with right setup, you can create interactive plots where you can zoom, pan, and more.

Why matplotlib?

- Built on Numpy Arrays and Python Dataframes
- integrates with pandas
- can create basic to advanced plots
- simple to use interface

Matplotlib workflow.

Data



1) create data

2) create plot
(figure)

3) plot data
(axes on figure)

4) customize plot

Importing and using Matplotlib

```
%matplotlib inline
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
import numpy as np
```

```
plt.plot()
```

```
plt.show()
```

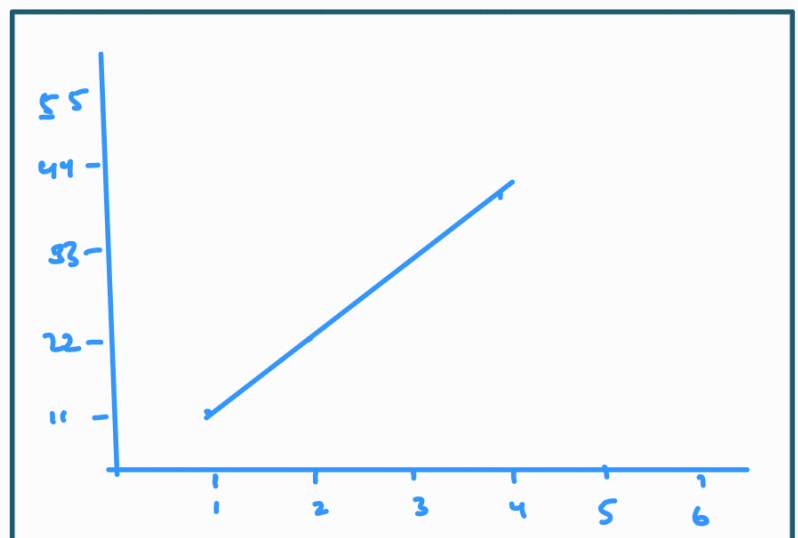
1) `plot(x, y)`

```
x = [1, 2, 3, 4]
```

```
y = [11, 22, 33, 44]
```

```
plt.plot(x, y)
```

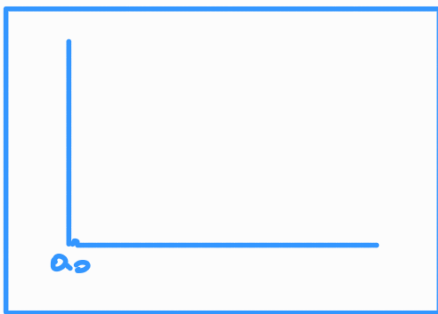
```
plt.show
```



In general try to use Object oriented interface over the pyplot interface

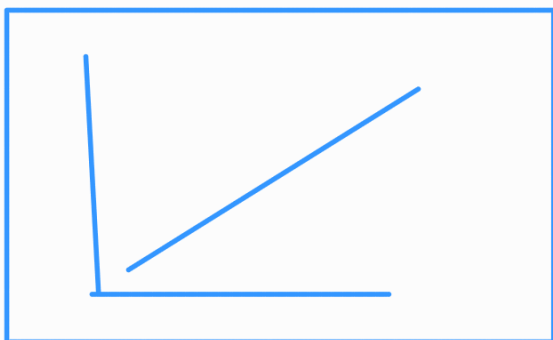
plotting methods

1) `fig = plt.figure()`
`ax = fig.add_subplot()`
`plt.show()`



2)

`fig = plt.figure()`
`ax = fig.add_axes([0.1, 0.1, 0.9, 0.9])`
`ax.plot(x, y)`
`plt.show()`



3) recommended way.

```
fig, ax = plt.subplots()
```

```
ax.plot(x, y)
```

```
plt.show()
```

```
type(fig), type(ax)
```

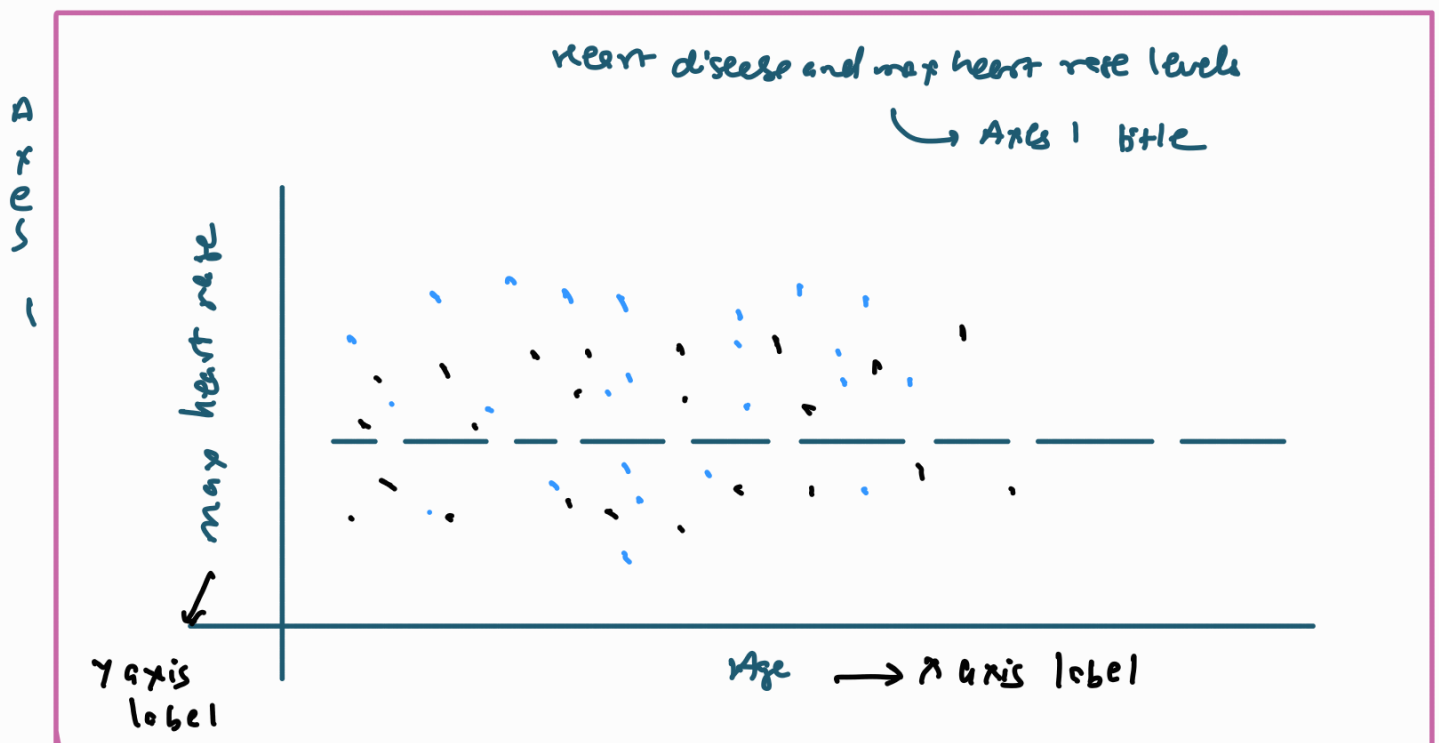
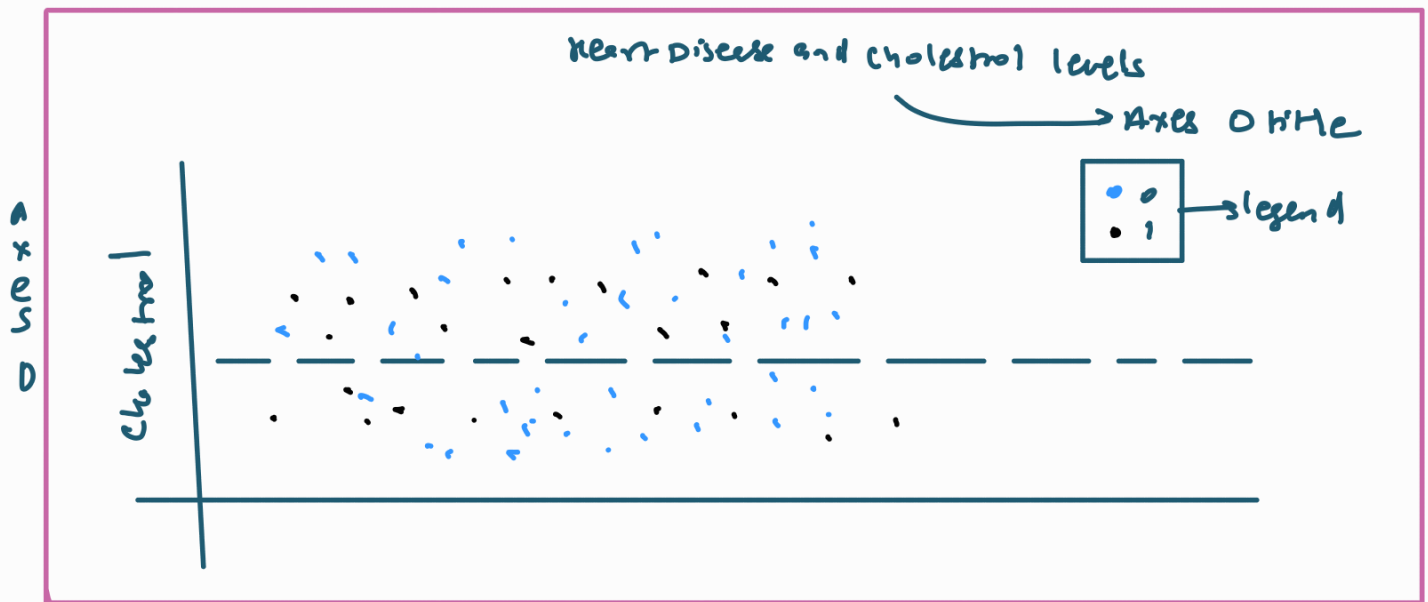
```
>> matplotlib.figure.Figure
```

```
matplotlib.axes._axes.Axes
```

Anatomy of a matplotlib figure.

figure

heart disease Analysis → Centred title



flow:

```
% matplotlib inline
```

```
import matplotlib.pyplot as plt
```

```
# prepare data
```

```
x = [1, 2, 3, 4]
```

```
y = [11, 22, 33, 44]
```

```
# setup plot
```

```
fig, ax = plt.subplots(figsize=(2, 2))
```

height, width

```
# plot data
```

```
ax.plot(x, y)
```

```
# customize plot
```

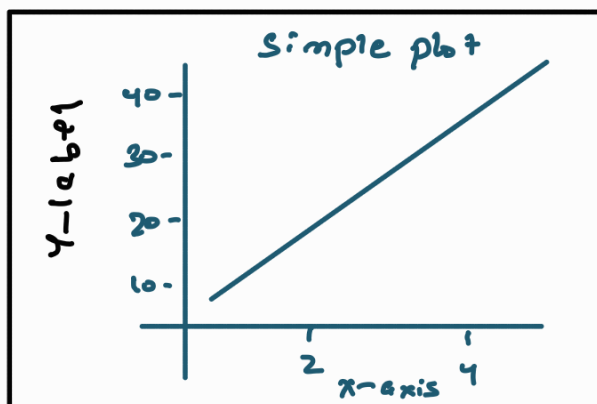
```
ax.set(title="Simple plot", xlabel="x-axis",  
       ylabel="y-axis")
```

```
# save plot
```

```
fig.savefig("path")
```

```
# print plot
```

```
plt.show
```



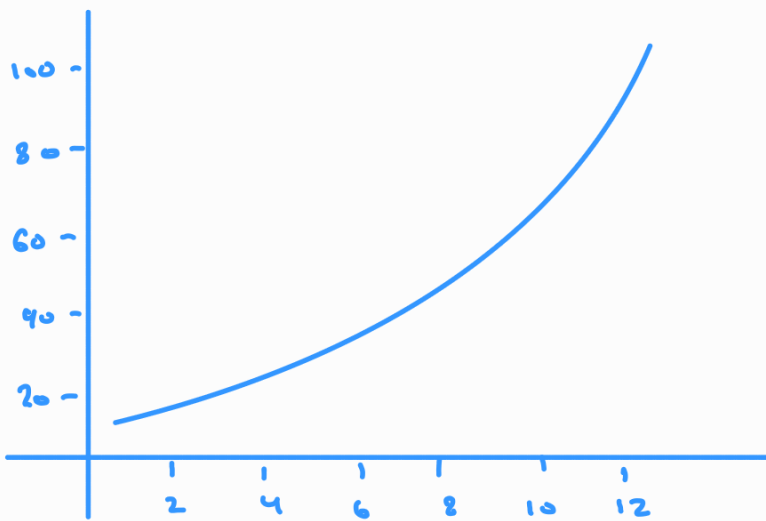
#making figures with numpy arrays

```
x = linspace(0, 10, 10)
```

```
fig, ax = plt.subplots(1)
```

```
ax.plot(x, x**2)
```

```
plt.show()
```

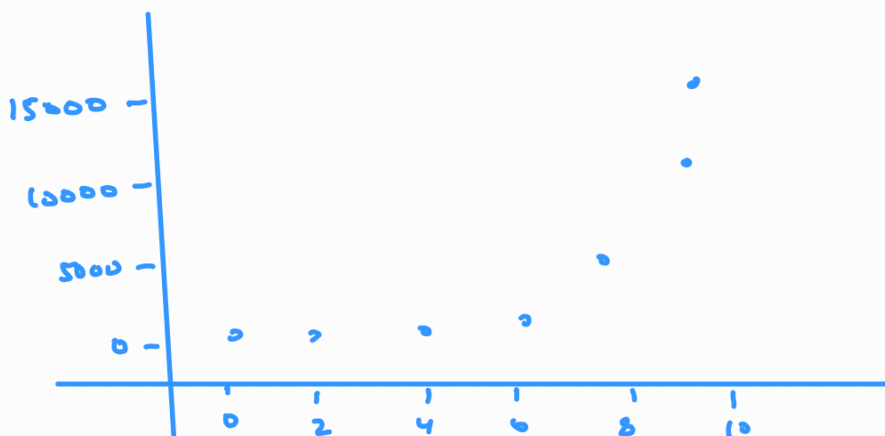


Scatter plot

```
fig, ax = plt.subplots(1)
```

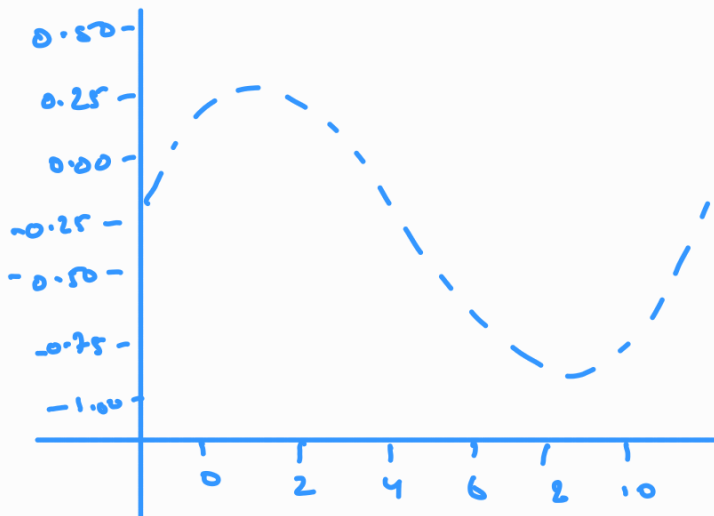
```
ax.scatter(x, exp(x))
```

```
plt.show()
```



```
fig, ax = plt.subplots()
```

```
ax.scatter(x, np.sin(x));
```



make a plot from dictionary.

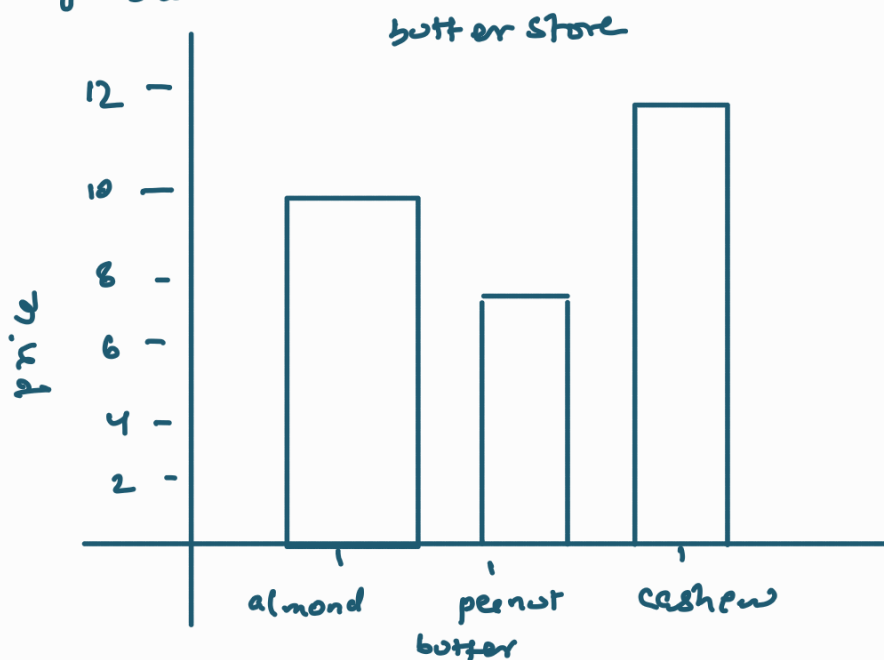
```
butter_prices = { "almond": 10,  
                  "peanut": 8,  
                  "cashew": 12 }
```

```
fig, ax = plt.subplots()
```

```
ax.bar(butter_prices.keys(), butter_prices.values)
```

```
ax.set(title = "butter store", xlabel = "butter", ylabel = "price")
```

```
plt.show()
```



horizontal bars

```
butter_prices = { "almond": 10,  
                  "pecan": 8,  
                  "cashew": 12 }
```

```
fig, ax = plt.subplots()
```

```
→ ax.barh(butter_prices.keys(), butter_prices.values)  (# works)
```

```
→ ax.barh(list(butter_prices.keys()), list(butter_prices.values))
```

```
ax.set(title = "butter store", xlabel = "butter", ylabel = "price")
```

```
plt.show()
```

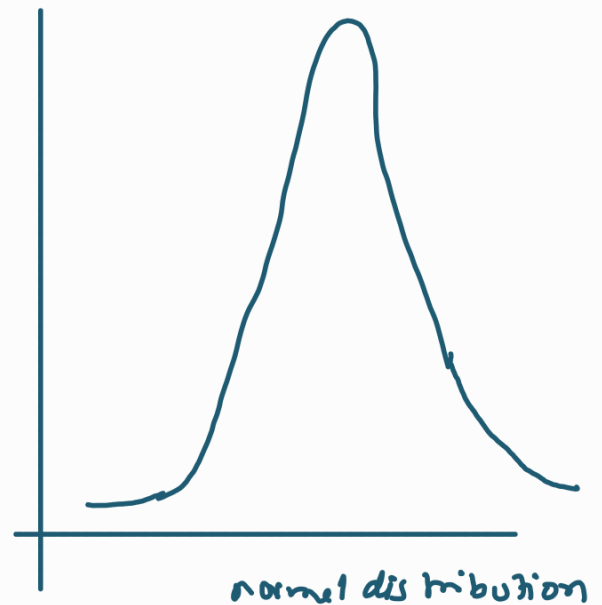
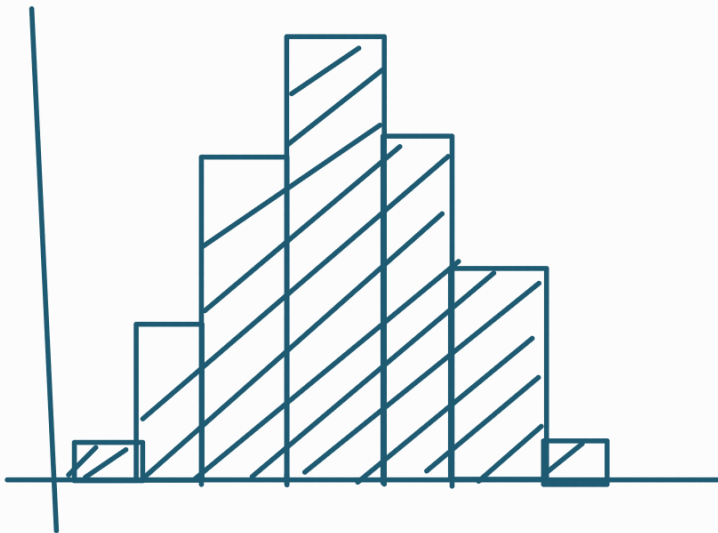
histograms and subplots

```
x = np.random.randn(1000)
```

returns a sample from std normal distribution

```
fig, ax = plt.subplots()
```

```
ax.hist()
```



Subplots.

↳ 2 options

1) option 1

```
fig, (ax1, ax2), (ax3, ax4) = plt.subplots (nrows=2,  
                                              ncols=2,  
                                              figsize=(10, 5))
```

↳ plotting in diff axes

```
ax1.plot(x, y/2)
```

```
ax2.scatter ( np.random.random(10), np.random.random(10) )
```

```
ax3.bar( butter_prices.keys(), butter_prices.values() )
```

```
ax4.hist( butter_prices.values() )
```

```
plt.show()
```

2) option 2.

↳ using indexes

```
fig, (ax1, ax2), (ax3, ax4) = plt.subplots (nrows=2,  
                                              ncols=2,  
                                              figsize=(10, 5))
```

↳ plotting in diff axes

```
ax[0,0].plot(x, y/2)
```

```
ax[0,1].scatter ( np.random.random(10), np.random.random(10) )
```

```
ax[1,0].bar( butter_prices.keys(), butter_prices.values() )
```

```
ax[1,1].hist( butter_prices.values() )
```

```
plt.show()
```

Regex

abc... letters

123... digits

\d any digit

\D any non-digit char

. any char

\. dot character

[abc] only abc

[^abc] apart from abc

[a-z] from a to z

[0-9] from 0-9

\w any alphanumeric char

\W any non-alphanumeric char

{n} n repetitions

{m,n} m to n repetitions

*

0 or more repetitions

+

1 or more repetitions

?

optional character

\s any whitespace

\S any non-whitespace

^...\$ starts and ends

(...)

capture group

(a(b))

capture sub-group

(.*)

capture all

Cars1def) matches abc or def

Plotting from Pandas Dataframes.

```
import pandas as pd
```

```
carsales = pd.read_csv("car-sales.csv")
```

removing \$ and converting to numeric in Price column

```
carsales["Price"] = pd.to_numeric(carsales["Price"].str.replace(
    '[\$,]', '', regex=True))
```

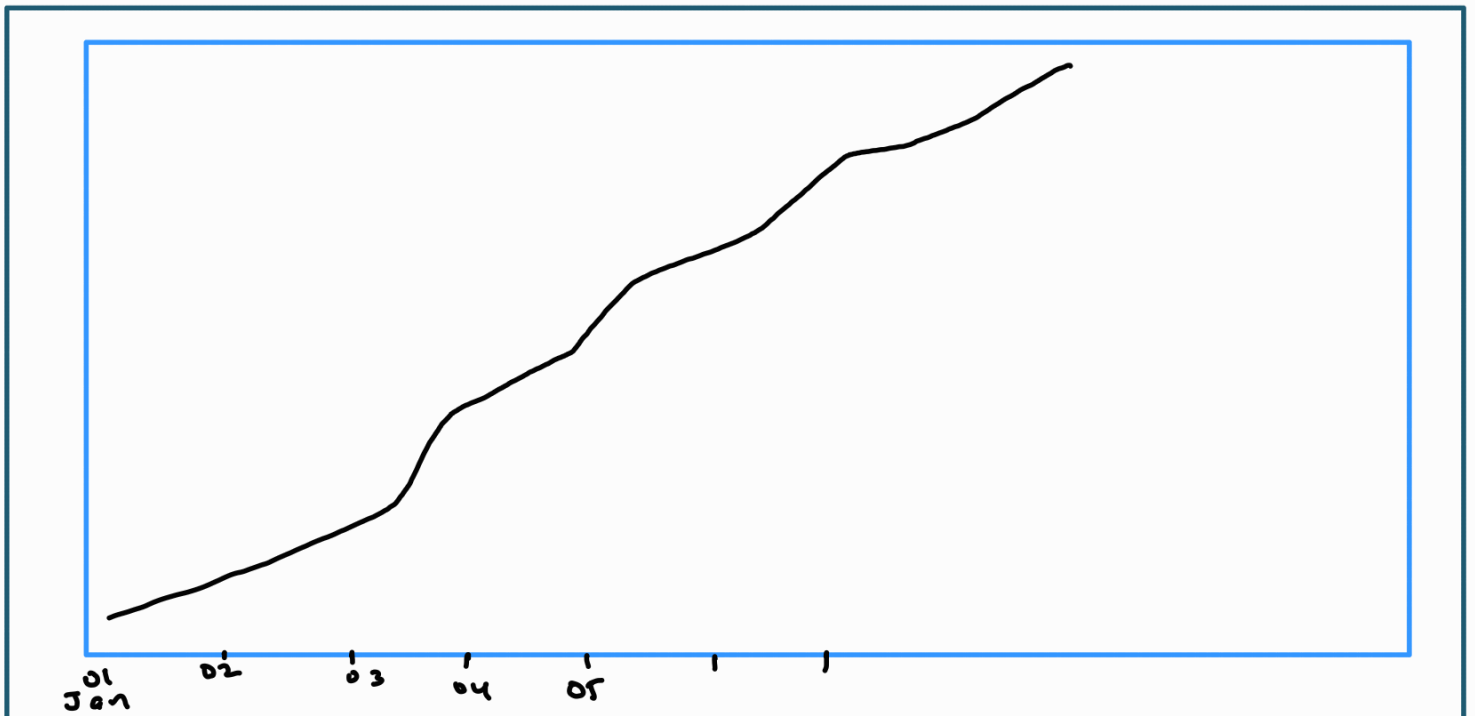
```
carsales["SaleDate"] = pd.date_range("01/01/2023", periods=len(carsales))
```

```
carsales["TotalSales"] = carsales["Price"].cumsum().astype(int)
```

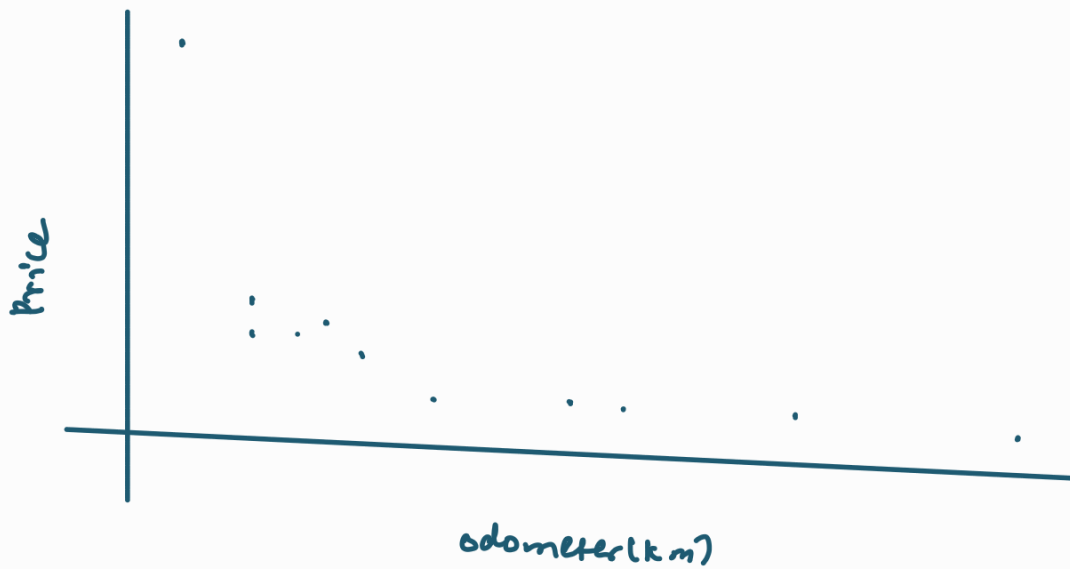
↳ optional to
convert float point

```
carsales.plot(x="SaleDate", y="TotalSales")
```

```
plt.show()
```



```
carsales.plot(x="odometer", y="price", kind='scatter')
```



eg 2

```
np.random.seed(0)
x = np.random.rand(10,4)
df = pd.DataFrame(x, columns = ['a', 'b', 'c', 'd'])
df.plot.bar()      (or) df.plot(kind="bar")
plt.show()
```

eg 3

```
carsales.plot(x="Make", y="Odometer (km)", kind="bar")
plt.show()
```

eg:- histogram of odometer

```
carsales["odometer (km)"].plot(kind="hist")
```

(or)

```
carsales["odometer (km)"].plot.hist(bins=5)
```

on heart disease csv

```
heart_disease = pd.read_csv("heart-disease.csv")
```

```
heart_disease["age"].plot.hist(color="skyblue")
```

```
plt.show()
```

→ Subplots.

```
heart_disease.plot.hist(figsize=(2,2), subplots=True)
```

↳ w x h

↳ each plot separately

OO method with pyplot

```
fig, ax = plt.subplots(figsize=(10,6))
```

```
over_50.plot(kind="scatter", x="age", y="chol", c="target", ax=ax)
```

```
ax.set_xlim([45,100])
```

```
plt.show()
```

OO method with scratch.

```
fig, ax = plt.subplots(figsize=(10,6))
```

#plot the data

```
ax = ax.scatter(x=over_50["age"],  
               y=over_50["chol"],  
               c=over_50["target"])
```

customize

```
ax.set(title="heart disease with cholesterol",  
       xlabel="age",  
       ylabel="cholesterol")
```

add a legend

```
ax.legend(*scatter.legend_elements(), title="target")  
  
ax.axhline(over_50["chol"].mean(), linestyle="--")  
  
plt.show()
```

④ we can customise plot colours

→ plt.style.available

→ plt.style.use("seaborn-v0_2")

→ cmap('')
 ↳ summer, winter, autumn, spring

→ Set_xlim(l1)
 ↳ set limits

set_ylim(l1)

saving and sharing the plots.

→ right click → save image as

(or)

→ fig.savefig("filename.png")

reset the plot

fig, ax = plt.subplots()

plt.show()

or) plt.clf()